Air Quality Robot

By Andy Liu A17112518 anl046@ucsd.edu, Jeffrey Lee A17442839 jsl019@ucsd.edu

Our motivation for the air quality robot is to help create an environment that is healthier for people. In light of the pandemic and recent natural disasters, the need for good air circulation and purification has been increasing. People are more aware of the air quality around them, and the impact that bad air quality can have. With this robot, we can help increase air circulation and lower the risk of diseases like Covid-19 from spreading, mitigate allergic reactions, and assist with any other risks to respiratory health from particles in the air. This is an autonomous robot that can monitor and cleanse the air, acting like a Roomba for the air.

Related Work

A big part of our project is being able to map out a room with an autonomous robot. There are plenty of people who have tried to do something similar. For example, Kai Nakamura made an autonomous mapping robot using SLAM. We are considering doing this, but with a lidar sensor. If we're able to get a decent mapping even with different sensors, that will be a big win for us.

In addition to autonomous navigation, we have found that negative ion generators have been used to improve air quality in various demonstrations such as this video showing that it can be effective at reducing the concentration of smoke in the air. There are also many examples online of people building automatic air monitoring systems or air conditioning systems using similar sensors and actuators that we are using (for example, in this video they apply similar sensors and actuators with a similar goal in mind).

However, we have not found many projects in which both autonomous navigation and automatic air monitoring/condition systems are integrated together, and this will likely be a major challenge as we put both components of the project together.

Development

HW Component

Listed below are the hardware components used in the project and where we sourced them:

- KBT 12V 5200mAh Rechargeable Li-ion Battery (Amazon)
- Adafruit PM2.5 Air Quality Sensor and Breadboard Adapter Kit - PMS5003 (Parts List)
- GLOGLOW Ionizer Generator Module (Amazon)
- PANO-MOUNTS 12V DC Fan (Amazon)
- The perseids DIY Robot Smart Car Chassis Kit (Amazon)
    - 1 Chassis
    - 4 Wheels
    - 4 DC Motors
    - 4 Speed Encoders
- ELEGOO DHT11 Digital Temperature & Humidity Sensor Module (37 in 1 Sensor Kit)
- ELEGOO Relay Module (37 in 1 Sensor Kit)
- 2 ELEGOO HC-SR04 Ultrasonic Sensor Modules (37 in 1 Sensor Kit)
- BOJACK L298N Motor DC Dual H-Bridge Motor Driver Controller Board Module Stepper (Amazon)

- HiLetgo CP2102 USB 2.0 to TTL Module Serial Converter Adapter Module USB to TTL Downloader with Jumper Wires (Amazon)

The main principles behind our hardware design choices focused on portability, cost effectiveness, and minimalism. Rather than have an entire robot car starter kit provided, we settled on the bare essentials for our robocar to ensure that it only had what was needed (chassis, motors, and wheels) for our project. In order to enable navigation, we used the provided ultrasonic sensor so that we would be able to map and detect objects in front of the car to enable its autonomous movement. The L298N motor driver was used for actuating motors as we desired, allowing us to control and actuate the motors with a 12V supply from the Lithium Ion battery. In addition to powering the motors, we needed the 12V to power both the fan and negative ion generator. Both fan and the ion generator module were controlled via the relay modules, allowing us to control the 12V input into those actuators via our Raspberry Pi 5. Finally, we have the DHT11 temperature and humidity sensor along with the PMS5003 PM2.5 air particle sensor, these are used in our software component in which we use to inform our actuators (fan and negative ion generator) on when and how to activate to interact with the environment.

Therefore, our system can be simplified to the following: ultrasonic sensor reads distance, based on those measurements motors help robot map and navigate, during this air quality measurements are stored, based on those measurements the robocar will navigate to spots to revisit and blow a fan on locations with high temperature and release negative ions at locations with high particle concentrations.

The reason for selecting a robocar (motors, chassis, wheels) was under the idea of expanding the range of air purifiers and air conditioning systems without increasing their cost. The biggest limitation to many existing systems is that to expand their range, significant costs must be incurred. For instance, the most effective air purifiers with extensive range are unaffordable to most. Expanding the range by making the device portable and navigate autonomously increases convenience while maintaining cost effectiveness and portability. This was the same reason for which we chose to use an ultrasonic sensor, however this ultimately backfired on us as the navigation with this sensor (and not a Lidar one or camera one) ultimately made it difficult for our algorithm to map.

The reason for selecting negative ion generators was due to their ability to ionize the air, generate negative ions, and then for us to release them via the fan into the air, to which they would attach to air particles and pull them out of the air and thereby improving air quality. In this scenario, not only would the fan be improving air circulation, but it would also have a convenient and portable means to purify the air without needing a filter to be replaced repeatedly (this was intended with cost effectiveness and portability in mind).

SW Component

- Raspberry Pi5 on aarch64 linux kernel
- Python 3.13
    - Matplotlib
    - NumPy
    - Thread
    - Time
    - Math
    - BreezySLAM
- C
- Wiring Pi

We used C to interface with the hardware and Python3.3 to write our main software. To use the sensors in Python, we had to create Python wrapper classes using ctypes, a foreign function library. This was possible through a library file that we compiled using our C code, like this:

```
gcc -shared -o fan_lib.so fan.c
```

For the actual Python class, we had an init for each sensor. Ctypes allowed us to pass by reference variables, which we passed into the C function. We wrapped C functions in Python like this:

```
Def start_fan(self):
    self.fan_lib.start_fan()
```

Another library we used was BreezySLAM, a lightweight Python library that helped us map out the room with our car. Initially, we thought we would be able to implement SLAM directly and avoid using Python, but that was a lot more complicated than we thought it would be. With the SLAM library, all we have to do is pass in sensor values for the SLAM function to work.

A SLAM (Simultaneous Localization And Mapping) algorithm was needed for us to navigate an unknown room. While the complexity of this algorithm is hidden behind the library, we have a very rudimentary understanding of this algorithm. There are different types of SLAM algorithms, depending on the sensors used. Popular SLAM algorithms are visual SLAM, which is done using a camera, and lidar SLAM, which is done using a lidar sensor. In our case, we used an ultrasonic sensor that imitated a lidar sensor. As far as the actual algorithm for SLAM, it is far beyond the scope of what we can implement in just a few weeks. At the bare minimum, it requires a strong foundation in linear algebra.

One challenge when working with BreezySLAM, and just SLAM in general, is that we are using an ultrasonic sensor instead of a Lidar sensor. The main difference is that Lidar gives a 360 degree view of the world, while ultrasonic gives a narrow view. BreezySLAM only supports Lidar, so we had to imitate a Lidar sensor with our ultrasonic sensor by inheriting the Laser(Lidar) class in BreezySLAM. Another difficulty was orienting and calibrating the position of the car without using an accelerometer. We were unable to implement an accelerometer as by the time we realized through our research it would be necessary for position tracking, it would have taken too much time to implement the accelerometer due to the complexity of how data is to be read from it. We had to manually test the speed of the car, and then pass it into an odometry function which calculated how far we had gone. After testing, we saw that our robot was moving at 300 mm/s, so we passed that into the odometry function. We also had to calibrate the direction(NorthEastSouthWest) of our robot after it turned. We did this by adding 1 to a direction variable, and modding it by 4 to get our orientation of the world. Because we hardcoded the speed, and because the ultrasonic sensor is so limiting, our mapping of the world with BreezySLAM is not very accurate.

We also had several macros for our robot car. In general, when navigating, we wanted to make sure our robot could avoid obstacles reasonably well. The ultrasonic sensor was useful here. We set the distance threshold for the ultrasonic sensor reading to be 250 centimeters. After testing, this value allowed the robot to move as close to an object as possible while minimizing collisions. For the speed of the car, it moved straight at a speed of 75 and turned at a speed of 150. These numbers are out of 255, which is the max speed we can set it to. As stated above, the forward speed is actually 300 mm/s, while the turning speed would be 600 mm/s. We opted to make the turning speed faster to reduce drift in measurement because we couldn't measure distance traveled when turning. It also helped with avoiding obstacles, since the robot would run into obstacles in the middle of turning to when the turning speed was also 75.

In our main file, we run our SLAM function, gathering data about temperature, humidity, and air quality at each point in the map. We keep track of the points that we want to revisit in an array. At the end of the mapping, we clean up the area by navigating back and actuating the fan and negative ion generator.

The main function looks like this:

```
Main():
    startSLAM()
    wait()
    stopSLAM()
    visualizeMap()
    cleanUp()
```

The actual function is more complex, but that is the general gist of the code.

To actually run the car and move over the room, we decided to add a timeout that would allow the car to move right backwards every once in a while. Initially, we wanted to make the car spiral, but this was harder than we thought it would be. Because our mapping is subpar, it is hard to know how far we've gone in one direction, and we can't adjust how far to move in each direction, especially if the room size varies. Instead, we added a random turn every 50 cycles of the loop. Based on our tests, 50 was the best number for it to get unstuck. This is useful both for being able to "randomly" navigate a room, and for getting unstuck when the car fails to avoid a collision. This way, it can eventually unstick itself instead of driving onto a wall indefinitely.

One of our goals was to be able to find areas with high concentration of particles, or higher temperatures and humidity. We set our threshold for these values at each point to be 15 degrees Celsius for temperature, 60% for humidity, and 350 um/0.1L. We decided on temperature based on the temperatures and humidity that we preferred. For the air particles, we noticed that it consistently read above 300 in the stuffy room we tested in, so we set the number to be 350.

In order to navigate back to areas that we marked above a certain threshold, I moved to each point linearly. I thought this would be close to optimal, because each location stored in the array should be closest to each other, since we're adding locations linearly. To actually navigate to the points, I decided between mapping algorithms like Djijkstras and A*. In the end, I decided to write a brute-force algorithm to go back to each location. I move to the x position, then to the y position. It is suboptimal, but it made testing and implementing a lot easier given our timeframe.

The code for navigating backwards looks like this:

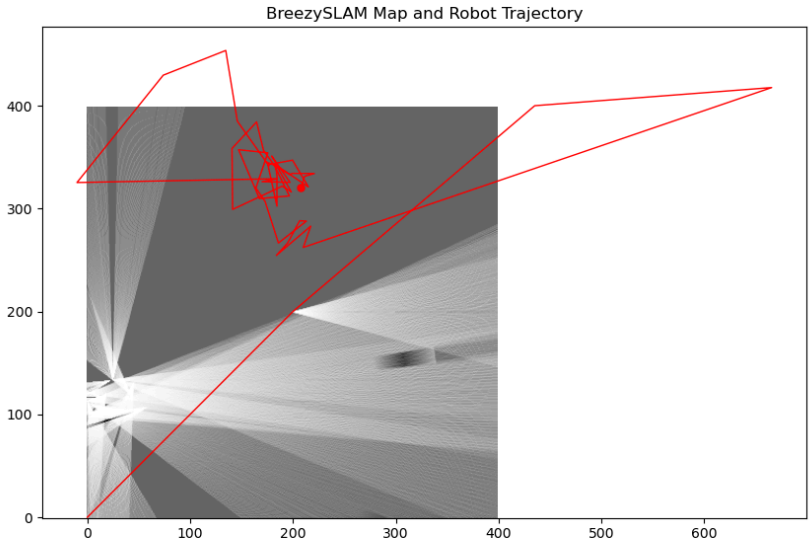
```
Def navBack():
    X, Y = current position()
    While ABS(X- targetX) < 5:
        If X < targetX:
            Orient direction
            Navigate backwards
        Else:
            Orient direction
            Navigate forwards
    While ABS(Y- targetY) < 5:
        If Y < targetY:
            Orient direction
            Navigate upwards
        Else:
            Orient direction
            Navigate downwards
```

However, this did not work too well because of our reliance on the initial mapping and localization to be accurate. So instead, we pivoted and actuated if we had stored enough data points above a certain threshold. This makes sense, because we would clear out the area if the temperature readings or air particles were too high, leading to better overall air quality. We chose 100 as the threshold to account for erroneous data.

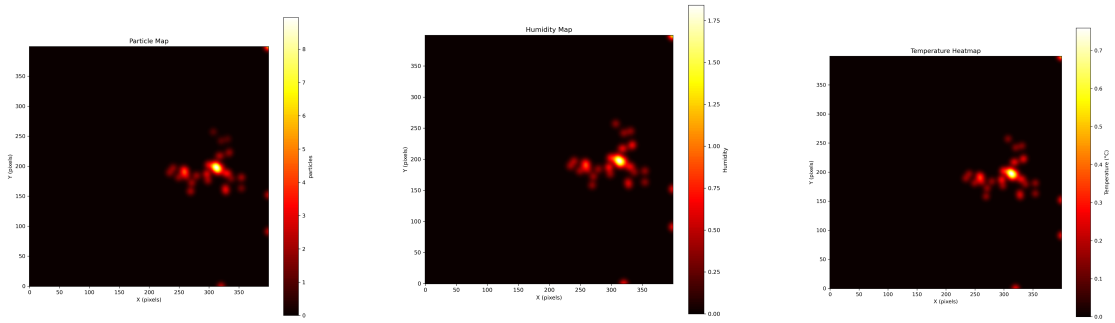
```
If needToRevisit > 100:
        Navigate
        Actuate Fan
        Stop Navigation
        Stop Fan
```

One more component that was implemented is a visualization tool. We use matplotlib and numpy for this, since it makes representing the data that we gather easy. Since the data we gathered from the mapping was not very accurate, the images were also hard to visualize. We used a gaussian filter to help enhance the image generated by removing noise and detail.

Experiments



The red lines show the path the robot car took. The gray is supposed to be a mapping of our world, but it is clearly inaccurate. We ran the mapping algorithm alot to finetune the numbers, and this was the best mapping we could get after tweaking parameters and algorithms.



These are the temperature, humidity, and particle maps. They all look similar, which is not too surprising given that our mapping is inaccurate, and we needed a gaussian filter to fine tune the details.

When doing our experiments and measuring the sensors, we found again and again that the sensors were not too accurate. They often had erroneous data, but we found that the sensors did follow general trends. For example, ultrasonic sensors would usually decrease the value the closer it was to an obstaclel, but sometimes it would give a high number even when right against an obstacle.

When checking the heat, temperature, and air particle again after actuating our fan, we found that it did not do anything meaningful to the data we collected. Unfortunately, one small fan just isn't enough to reduce the air particles in a large room over the period of time which we ran it. We did notice a few times that the temperature would change in the isolated area when we ran the fan, but we weren't sure, since the data would also just occasionally give out junk values. We could not establish a trend. However, the hope is that in the future, we could use additional more fine-grained sensors and science to purify the air properly.

Conclusion

We wanted to create an autonomous robot car that is able to avoid obstacles, while mapping out a room and purifying it. We were able to meet some of these goals, but not entirely because of our limited sensors. While the car is able to avoid some obstacles, it occasionally gets stuck. It also does a poor job of mapping out the room because of its limited sensors. We needed to use a lidar sensor, which has a 360 degree view of the world, but our ultrasonic sensor only has a 15 degree view of the world! We also struggled with race conditions with the sensors when running threads, which made us decide to abandon our cleanup code since it wouldn't work properly anyway with how inaccurate the mapping was. However, as a proof of concept, we showed that we're able to get some mapping of the room, and the robot is able to navigate and actuate fans in areas with low quality air. The robot does not go back to the exact same spot, but if our mapping and localization was better, it would do a better job. The main takeaway we had was that in order to fully implement localization, we would need higher quality sensors that could take advantage of more complex algorithms.

Sources/Citations:

https://claude.ai/share/ddb7fc47-9824-4079-9099-824739a1a959
https://docs.python.org/3/library/ctypes.html
https://stackoverflow.com/questions/5081875/ctypes-beginner